A Frame-Based Service Agent with Tool-Call Auditing for EgoLink 2026

SJTU MediaLab Team
SJTU MediaLab
Shanghai, China
lance_dpn@sjtu.edu.cn

Abstract

EgoLink 2026 Track 2 evaluates service agents in egocentric video environments where a simulated user issues multi-turn requests and the service agent must combine visual grounding with official tool calls. This report describes our submitted system, which keeps the official user simulator and tool/database sandbox unchanged, while replacing the service side with a frame-grounded agent. Videos are uniformly sampled into timestamped image frames and attached to an OpenAI Responses-style service model only when visual grounding is likely to be needed. The service model uses scenario-aware instructions to convert visual hypotheses into canonical database entities through read-only tools, then performs state-changing actions or final calculations through official tools. A lightweight correction agent audits proposed tool batches and final replies for schema, evidence, state-change, and calculation consistency. No backbone model is trained or fine-tuned; all gains come from orchestration, prompting, frame sampling, recovery controls, and preflight auditing.

CCS Concepts

• **Computing methodologies** → **Artificial intelligence**; *Computer vision tasks*; • **Human-centered computing** → *Interactive systems and tools*.

Keywords

egocentric video understanding, tool-using agents, service agent, benchmark, multimodal reasoning

1 Introduction

EgoLink 2026 Track 2 is an egocentric-video benchmark for service agents that must complete realistic user requests through multi-turn interaction and tool use [1]. Each task is embedded in a scenario such as retail, kitchen, restaurant, or ordering. The user side is simulated by an official LLM-based user agent, while the service side is the participant-controlled component. The service agent must identify visually grounded referents from the video, interact naturally with the user, call tools in a strict JSON format, and update the official scenario database when requested.

Our implementation repository is `Lance-dpn/EgoBench`: https://github.com/Lance-dpn/EgoBench [2]. During the evaluation period the repository is private and will be made public after the evaluation phase. The submitted system is implemented under `experiments/gpt55_frame_service_runner`. It preserves the official interaction sandbox and replaces only the service-agent inference path with a frame-based multimodal service agent and an optional correction pass.

2 Benchmark Environment

The official sandbox is organized around scenario files, tool schemas, database implementations, and a multi-agent runner. The runner initializes a fresh domain database for each task, builds the simulated user's prompt from the task instruction and video description, executes alternating user and service turns, parses service-agent tool calls, executes official tools, and writes one trajectory file per scenario. The tool directories define the public operation surface for each domain: retail carts and product facts, kitchen inventory and recipes, restaurant orders and menu facts, and multi-restaurant ordering.

The official evaluator computes three complementary metrics. The tool-based metric checks whether reference tool calls appear in the interaction log, with scenario-specific fuzzy matching for item names and parameter normalization. Extra tool calls are tolerated, so this metric rewards recovering the required process rather than minimizing every call. The result-based metric replays the ground-truth call sequence and the submitted call sequence from the same initial database, then compares normalized database-state hashes. Joint success requires both tool-based and result-based success. This makes final correctness and process faithfulness both important.

The final set considered in this report contains five scenarios: `retail6`, `retail10`, `kitchen4`, `restaurant5`, and `order2`. The service agent does not directly read hidden answers or use final-evaluation scenario annotations as evidence. It observes only the user dialogue, sampled video frames supplied by our runner, official tool descriptions, and executed tool results.

3 Method

3.1 Frame-Grounded Service Agent

The official runner supports video inputs for some service models, but our system uses sampled frames as the visual interface. The module `frame_sampler.py` uses `ffprobe` to estimate duration and `ffmpeg` to extract uniformly spaced JPEG frames. A cache key includes the video path, file metadata, duration, sampling rate, image scale, quality, maximum frame count, rotation, and timestamps. This prevents repeated video decoding across runs while keeping

the cache invalidated when the source video or sampling policy changes.

The service agent receives text conversation history and, when appropriate, timestamped frame images encoded as data URLs. We use a lazy attachment policy. If the latest user message contains visual language such as pointing, color, position, menu, shelf, region, or visible-object references, frames are attached on the first service call for that turn. Otherwise the model starts text-only and may output exactly `NEED_VISUAL_CONTEXT`; the runner then retries the same turn with frames attached. This policy reduces large image payloads while preserving access to visual context for visually grounded requests.

3.2 Scenario-Aware Tool Reasoning

The service prompt is intentionally strict. It requires tool calls to be exactly one JSON array and forbids prose mixed with tool-call JSON. It also separates visual evidence from database evidence: frames are used to form hypotheses about products, dishes, ingredients, actions, regions, and spatial relations, while official tools are authoritative for canonical names, prices, nutrition, allergens, tax, discounts, orders, carts, shopping lists, and calculations.

Each scenario has additional rules. Retail emphasizes canonicalizing visible products through product-name lookups and reusing returned official names. Kitchen emphasizes matching visible ingredients against official ingredient names and avoiding recipe guesses from vision alone. Restaurant tasks emphasize menu orientation, category localization, pointing order, and bounded ranking inside localized sections. Order tasks emphasize restaurant selection, set-meal handling, and stable menu-section constraints. Across domains, the agent uses progressive filtering: locate the visual or dialogue boundary, retrieve the candidate set, apply branch or attribute conditions, rank only the surviving candidates, and mutate state only after the active target is supported by tool results.

3.3 Correction Agent and Guards

The correction component audits proposed service-agent outputs before they are executed or shown to the simulated user. It does not receive images and is explicitly instructed not to judge visual target correctness. Instead, it checks tool names, parameter schemas, branch evidence, canonical-name support, state-changing mutations, final calculations, and reply support. Read-only batches such as `find_*`, `get_*`, `list_*`, `tally_*`, and `compute_*` are automatically approved by default, because they are the mechanism by which visual hypotheses are canonicalized. State-changing calls such as `add_*`, `remove_*`, and `clear_*` are audited more strictly.

The runner also includes a repeat guard. If a state-changing call with the same tool name and parameters has already succeeded and the current user message does not explicitly ask for an additional or repeated change, the duplicate mutation is blocked and feedback is added to the service history. This reduces hash-damaging over-operation in multi-turn conversations.

4 Experimental Settings

Unless otherwise specified, the submitted runs use the full frame-based service agent with correction/preflight auditing.

4.1 Service Model

The service agent uses an OpenAI-Responses-compatible endpoint configured with `SERVICE_MODEL_NAME`, `SERVICE_API_KEY`, and `SERVICE_API_BASE_URL`, with `OPENAI_*` accepted as a generic compatibility fallback; default model `gpt-5.5`; temperature 0.0; reasoning effort `low`; and maximum output length 32768 tokens.

4.2 User Simulator

The simulated user follows the official prompt with a separate OpenAI-compatible chat-completions path; default model `qwen3.5-397b-a17b`; temperature 0.0; and both `--multi_agent_user` and `--summary_user` enabled.

4.3 Visual Input

Videos are uniformly sampled at 2 fps with maximum side length 1920 and JPEG quality 3; Responses image detail `high`; frame attachment policy `auto`; and at most 6 visual-context requests per user turn.

4.4 Interaction Budget

Each task allows up to 10 user turns, 12 inner service/tool rounds per user turn, and 200 total tool calls.

4.5 Correction Agent

The correction agent uses the Responses API and reuses the service model unless overridden; temperature 0.0; reasoning effort `medium`; at most 5 correction rounds; and automatic approval for read-only tool batches.

4.6 Retry Policy

Service calls allow 8 attempts with 30 second base delay, 180 second maximum delay, and a 300 second retry-after cap.

4.7 Training

No backbone or foundation model was trained or fine-tuned. The method is an inference-time system built from prompting, frame sampling, tool orchestration, retry/recovery logic, and preflight auditing.

5 Submission Package

The submitted artifacts contain trajectories for the five final scenarios: `retail6`, `retail10`, `kitchen4`, `restaurant5`, and `order2`. During development, local GT-based checks were used only as debugging tools for trajectory inspection and regression testing. They are not official benchmark labels and are not reported as official results in this technical report.

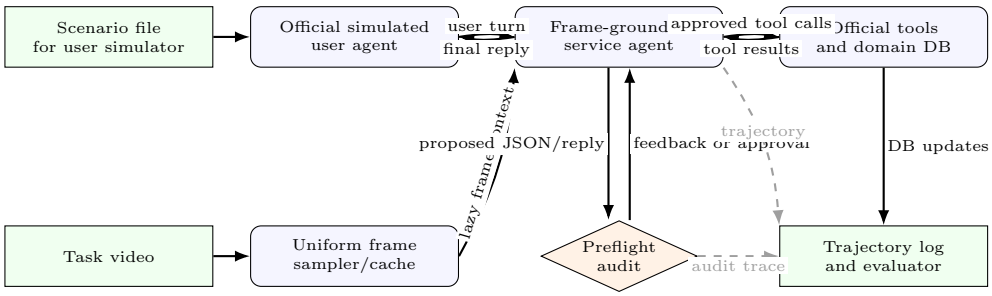

Figure 1: System overview. The official user simulator, tools, and databases remain unchanged. Our runner supplies sampled visual context to the service agent, audits proposed actions, executes approved official tools, and records trajectories for the official evaluator.

Figure 2: Frame-Grounded Service-Agent Inference
Inputs: scenario task, official tool schema, task video, and fresh domain database.

(1) Sample and cache timestamped frames at 2 fps.
(2) Initialize the official simulated user and service history.
(3) For each user turn up to the turn limit:
  (a) Generate the next simulated-user message.
  (b) Set *attachFrames* by the auto visual-reference policy.
  (c) For each inner service/tool round up to the tool-round limit:
    (i) Call the service model with dialogue and optional frames.
    (ii) If the model outputs `NEED_VISUAL_CONTEXT`, retry this turn with frames attached when allowed.
    (iii) If the model outputs strict JSON tool calls, audit the proposed batch, block duplicate successful mutations unless the user asks for more, execute approved calls through official tools, and append results to service history.
    (iv) Otherwise, audit the final natural-language reply, show an approved reply to the simulated user, and leave the inner loop.
  (d) Save the task trajectory checkpoint.
  (e) Stop the task if the simulated user outputs `STOP`.

6 Reproducibility

The system can be reproduced from a fresh checkout with standard Python dependencies, local videos, and API credentials for the service and simulated user models. The repository will be public after the evaluation phase.

Repeat the same command pattern for the other final scenarios by changing the scenario name and number: `retail10`, `kitchen4`, `restaurant5`, and `order2`. The runner writes task-level checkpoints under `results/<output_model_name>/`, so interrupted runs can be resumed with the same output model name and the `--resume` flag. After all five scenario files are generated, the official evaluation helper can be run as:

```
cd analysis_scripts
python evaluate_interaction.py \
  --model_name <team_or_run_name> \
  --num_samples 0
```

7 Limitations

The system is an inference-time orchestration method and inherits the strengths and weaknesses of the underlying foundation models. Uniform frame sampling may miss short-lived pointing or motion cues between sampled timestamps, and high resolution frame payloads can be large for long videos. The correction agent is deliberately scoped to official tool usage and database evidence; it does not verify whether the service model selected the correct visual object, category, or spatial referent from the frames. Future work could improve adaptive frame selection, explicit visual target localization, and domain-specific recovery for difficult menu and order tasks.

References

[1] EgoLink Organizers. 2026. EgoLink 2026 Track 2 Official Repository. https://github.com/ego-link/egolink2026. Accessed June 2026.
[2] Lance-dpn Team. 2026. EgoBench System Repository. https://github.com/Lance-dpn/EgoBench. Private during evaluation; to be made public after the evaluation phase.

```
# 1. Clone and create the runtime environment.
git clone https://github.com/Lance-dpn/EgoBench.git
cd EgoBench
conda create -n egolink python=3.10 -y
conda activate egolink
pip install -r requirements.txt

# 2. Make ffmpeg and ffprobe available, for example:
conda install -c conda-forge ffmpeg -y

# 3. Configure model endpoints and local video storage.
export SERVICE_MODEL_NAME=<service-model>
export SERVICE_API_KEY=<service-api-key>
export SERVICE_API_BASE_URL=<service-api-base-url>
export USER_MODEL_NAME=<user-model>
export USER_API_KEY=<user-api-key>
export USER_API_BASE_URL=<user-api-base-url>
export VIDEO_LOCAL_PATH=/path/to/egolink/videos

# 4. Run one final-evaluation scenario.
python -u experiments/gpt55_frame_service_runner/run_frame_agent.py \
  --scenario retail \
  --scenario_number 6 \
  --output_model_name <team_or_run_name> \
  --multi_agent_user \
  --summary_user \
  --frame_fps 2 \
  --frame_max_side 1920 \
  --image_detail high \
  --frame_attach_policy auto \
  --enable_correction_agent \
  --resume
```

Figure 3: README-style setup and representative command for one final-evaluation scenario.